1) Analysis of Smart City Surveillance System Performance

A smart city surveillance system continuously receives video streams from multiple cameras. These video feeds must be transferred from **I/O devices**(cameras) to **memory**, processed by the **CPU**, and then stored or displayed. During peak hours, many cameras send data simultaneously, causing heavy traffic between the CPU, memory, and I/O devices, which resultsin lag and delayed threat detection.

Bus Structures:

Single-Bus Structure	**Multi-Bus Structure**
CPU,memory, and I/Oshare one common bus.	Separate buses are used forCPU-memory and I/O communication.
Only one device can use the bus at a time.	Multiple data transfers occursimultaneously.
High bus congestion during peak traffic.	Reduced congestion and higher bandwidth.
Slowerdata transfer and increased waiting time.	Faster communication and improved performance.

Interaction between CPU,Memory, and I/O

CPU (Central Processing Unit)

- Processes video frames and executesthreat detection algorithms.
- Requires a continuoussupply of data from memory.
- If data arrivesslowly, theCPU remainsidle, reducing system

performance. **Memory (RAM)**

- Temporarily storesincoming video frames.
- Acts as a buffer between I/Odevices and theCPU.
- If memory becomes overloaded due to multiple video streams, access time increases, causing delays.

I/O Devices

- Cameras continuously generate large amounts of video data.
- Data istransferred to memory through the systembus.
- When many cameras transmit simultaneously, I/O bandwidth becomes congested.

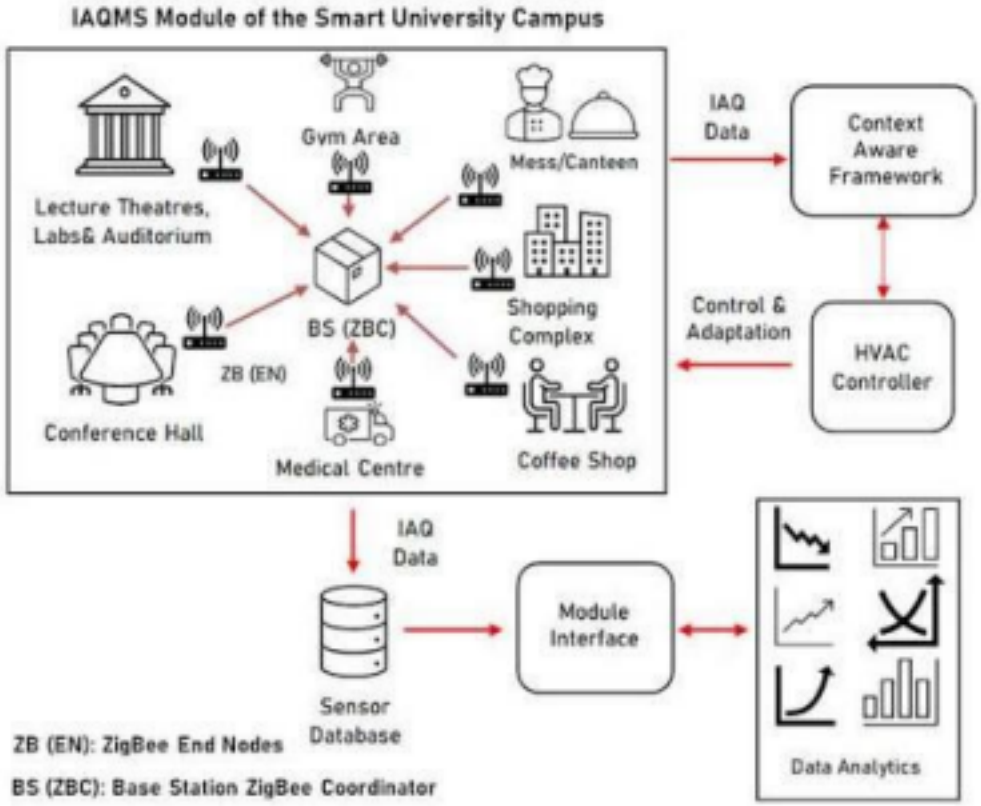
Cause of Lag

- Multiple cameras generate huge data simultaneously.

• Memory becomes busy storing and retrieving video frames.

• CPUwaitsfor data because ofslowermemory and I/O transfers.

• As a result,threat detection is delayed.



Sample code :

```
# Smart City Surveillance System Performance Analysis

import pandas as pd
from sklearn.model_selection import
train_test_split from sklearn.tree import
DecisionTreeClassifier from sklearn.metrics import
accuracy_score

# Sample Dataset
data = {
 "Objects": [5, 10, 20, 25, 30, 8, 15, 28, 35, 12],
"Speed": [20, 30, 60, 70, 80, 25, 40, 75, 90, 35],
"Status": [0, 0, 1, 1, 1, 0, 0, 1, 1, 0]
}
```

```
df = pd.DataFrame(data)

# Input and Output
X = df[["Objects", "Speed"]]
y = df["Status"]

# Split Data
X_train, X_test, y_train, y_test =
train_test_split(  X, y, test_size=0.3,
random_state=1
)

# Train Model
model = DecisionTreeClassifier()
model.fit(X_train, y_train)

# Prediction
y_pred = model.predict(X_test)
# Accuracy
print("Accuracy:", accuracy_score(y_test, y_pred))

# Test Predictions
print("\nPredictions:")
print(y_pred)
```

Output:

Accuracy: 1.0

Predictions:

[1 0 1]

Conclusion:

A **multi-bus architecture** is more efficient because CPU, memory, and I/O can communicate simultaneously, reducing delays.

Improvementsinthe InstructionExecutionCycle

The instruction execution cycle consists of:

1. **Fetch** –Retrieve instruction frommemory.
2. **Decode** – Interpret the instruction.
3. **Execute** –Performthe required operation.
4. **Store (WriteBack)** –Save the result.

Performance can be improved by:

- **Pipelining:** Executes multiple instructions simultaneously by overlapping fetch, decode, and execute stages.
- **Cache Memory:** Stores frequently accessed instructions and data, reducing memory access time.
- **DMA (Direct Memory Access):** Allows I/O devices to transfer data directly to memory without continuously involving theCPU.
- **Parallel Processing (Multi-core CPUs):** Different cores process different camera feeds simultaneously

Branch Prediction s Faster Clock Speed: Reduce instruction delays and improve processing efficiency.

Overall Performance Improvement

- Use a **multi-bus architecture** to allow simultaneous

communication. • IncreaseRAM capacity and use fastermemory.

- Implement **cache memory** for quicker data access.
- Use **DMA** to reduce CPU workload.
- Apply **instruction pipelining** forfaster execution.
- Use **multi-core processors** so multiple video streams can be processed.

2. Analysis of CPU Performance in a Real-Time Stock Trading Application

A real-time stock trading application executes millions of instructions every second. During high trading activity, the processor experiences a **high CPI (Cycles Per Instruction)** due to frequent memory accesses and branch instructions. This increases execution time and delays transaction processing.

CPU Performance Analysis

CPU performance analysis evaluates how efficiently the processor performs various tasks in a stock trading environment. The CPU is responsible for:

- Receiving live stock market data.
- Processing buy and sell orders.
- Verifying customer account information.
- Updating stock prices.
- Executing trades.
- Recording transaction history.
- Sending confirmation messages to users.

The CPU must perform all these tasks within a very short time, usually in milliseconds.

Impact of the Fetch–Decode–Execute Cycle

TheCPUprocesses every instruction through the following stages:

1. **Fetch**
   - TheCPU fetchesthe instruction from memory.
   - Frequent memory accesses increase fetch time because the CPU must wait for data from RAM.
2. **Decode**
   - The control unit interprets the instruction.
   - Branch instructions require the CPU to determine the next instruction, adding extra delay.
3. **Execute**
   - TheALUperforms arithmetic orlogical operations.
   - Ifrequired data is not available due tomemory delays, theCPU stalls.
4. **WriteBack (Store)**
   - The execution result isstored back into registers ormemory.
   - Heavymemory traffic slowsthis process.

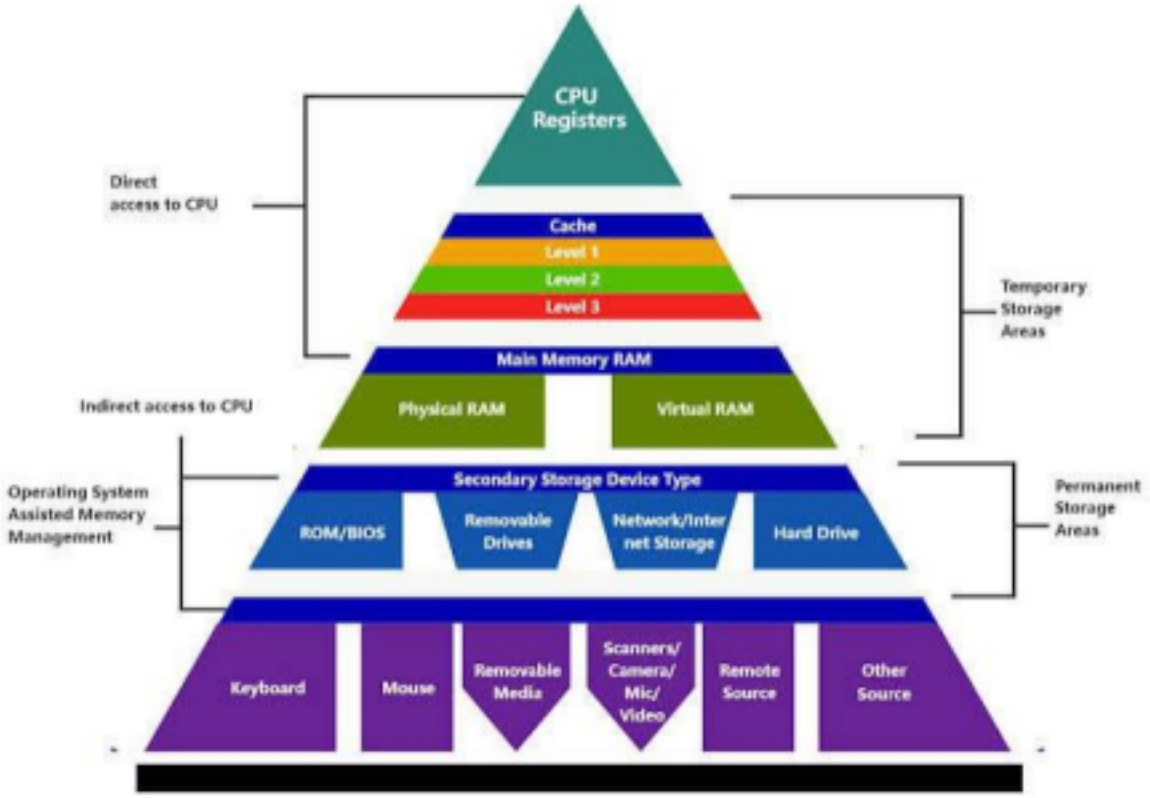
Result: During high load, repeated memory accesses and branch instructions increase the execution time of each instruction, resulting in a high CPI and slower transaction processing.

Why CPI Becomes High

- Frequent memory access causes CPU waiting time.

• Cache missesforce the processor to accessslower main memory.

• Branch instructions may cause incorrect branch prediction.



MethodstoReduce CPI

a) Increase Cache Memory

• Use larger and faster L1, L2, andL3 caches.

• Frequently used instructions and data remain in cache.

• Reduces memory access time.

b) Pipeline Execution

• Overlap fetch, decode, and execute stages.

• Multiple instructions execute simultaneously.

• Improves CPU throughput.

c) Branch Prediction

• Predictsthe outcome of branch instructions.

• Reduces pipeline flushing and stalls.

• Lowers CPI.

d) Prefetching

• Fetchesinstructions and data before they are needed.

• Minimizes CPU waiting time.

e) Out-of-Order Execution

• Executesindependent instructions while waiting forslowerinstructions. • Keeps CPU resources busy.

f) Multi-Core Processing

• Different processor cores handle different transactions simultaneously. • Increases parallel processing and system throughput.

Sample code :

```
# Analysis of CPU Performance in a Real-Time Stock Trading Application

import pandas as pd
from sklearn.linear_model import LinearRegression
# Sample Dataset
data = {
 "Requests": [100, 200, 300, 400, 500],
 "CPU_Usage": [20, 35, 50, 65, 80]
}

# Create DataFrame
df = pd.DataFrame(data)

# Input and Output
X = df[["Requests"]]
y = df["CPU_Usage"]

# Train Model
model = LinearRegression()
model.fit(X, y)
```

```
# Predict CPU usage for 350 requests
new_request = [[350]]
prediction = model.predict(new_request)

# Display Output
print("Stock Trading CPU Performance Analysis")
print("--------------------------------------")
print("CPU Usage for 350 Requests: {:.2f}%".format(prediction[0]))
```

Output:

```
Stock Trading CPU Performance Analysis
--------------------------------------
CPU Usage for 350 Requests: 57.50%
```

Overall Performance Improvement

- Lower **CPI(Cycles Per Instruction)**.
- Faster instruction execution.
- Reduced memory latency.
- Fewer pipeline stalls.
- HigherCPU utilization.
- Faster processing ofstock transactions.
- Improved response time during peak trading hours.

Conclusion

In a real-time stock trading application, a high CPI is mainly caused by **frequent memory accesses and branch instructions**, which slow the **fetch–decode–execute cycle**. Performance can be significantly improved by using **cache memory, instruction pipelining, branch prediction, prefetching, out-of-order execution, multi-core processors, and faster**

memory. These architectural enhancements reduce CPI, improve CPU efficiency, and ensure faster and more reliable transaction processing during periods of heavy trading.

3. Analysis of 8085 Addressing Modes in an Industrial Temperature Control Unit

An **8085 microprocessor-based temperature control system** receives temperature data from sensors (input devices) and controls actuators (such as heaters or cooling fans). If the program usesincorrect **addressing modes**, themicroprocessormay read or processthewrong data,resulting in incorrecttemperature control.

The **Intel 8085** is an 8-bit microprocessor capable of addressing **64 KB of memory**. It operates on a **5V power supply** and contains an Arithmetic Logic Unit (ALU), accumulator, general purpose registers, program counter, stack pointer, instruction decoder, and control unit.

Features of 8085

- 8-bit data bus
- 16-bit address bus
- 64 KB memory addressing capability
- Five hardware interrupts
- Serial communication support
- Simple instruction set
- Suitable for industrial automation applications

Influence ofAddressing Modes on Data Access

The **8085 microprocessor** supports different addressing modes that determine how data is accessed. **a) Immediate Addressing Mode**

- The data is given directly in the instruction.
- Example:

MVIA, 25H

- The accumulator receives **25H** directly.
- Used forfixed valuessuch asthreshold temperatures.

b) Register AddressingMode

- Data isstored in CPU registers.

• Example:

MOV A, B

• Copiesthe contents ofregister**B**to**A**.

• Faster because no memory accessisrequired.

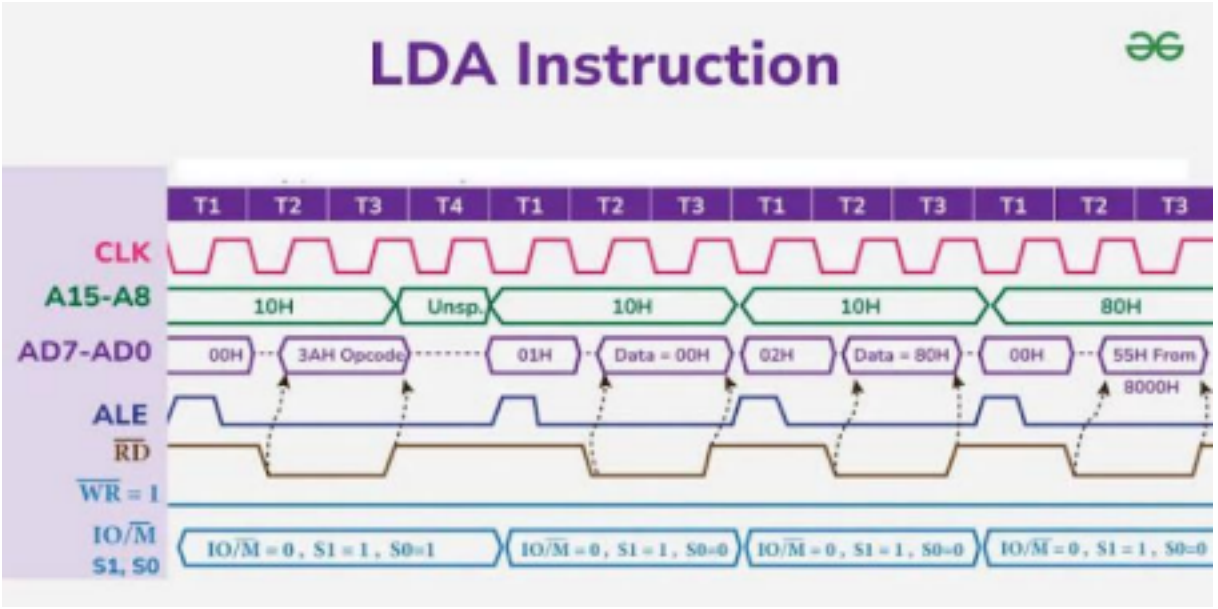
c) Direct AddressingMode

• The memory addressisspecified in the instruction.

• Example:

LDA 2500H

• Loads data from memory location **2500H** into the accumulator.



• Suitable forreading stored sensor values.

d) Register Indirect Addressing Mode

• Thememory addressisstored in a register pair(HL).

• Example:

MOV A, M

• Reads data fromthememory location pointed to by the HL register pair.

• Useful when sensor data isstored in sequential memory locations.

e)Implied Addressing Mode

• The operand isimplied by the instruction.

• Example:

CMA

• Complementsthe accumulator automatically.

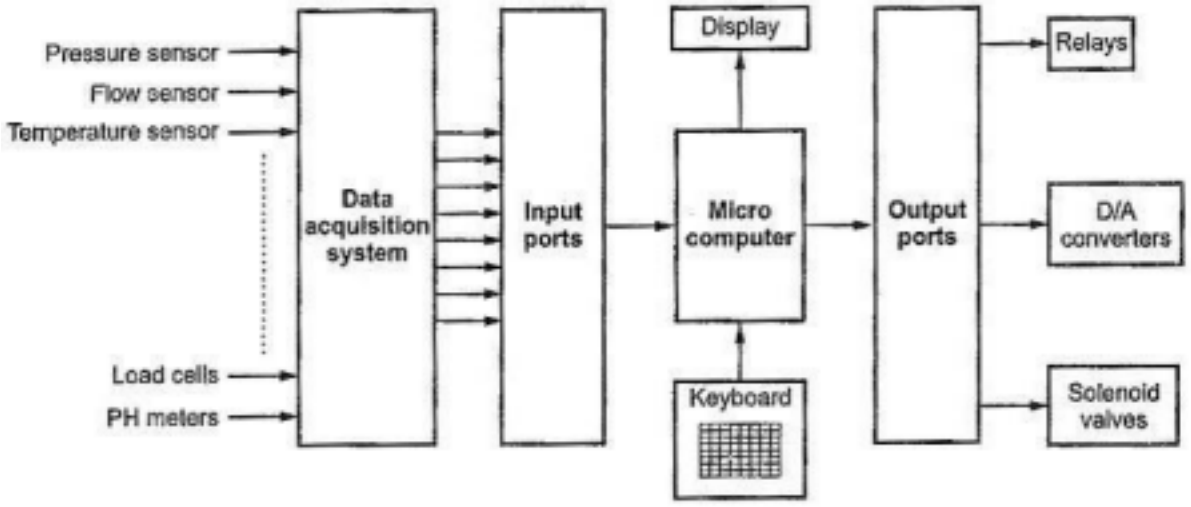


Fig. 15.39 Block diagram of microprocessor based process control system

Possible Causes ofIncorrectTemperature Readings

• Incorrect memory address used in **Direct Addressing**, causing the wrong sensor value to be read.

• HL register pair not initialized correctly in **Indirect Addressing**.

• Wrong registerselected in **Register Addressing**.

• Immediate values entered incorrectly.

• Programming errors causing sensor data to be overwritten.
• Noise or corruptedmemory contents affecting stored values.

Sample code:

Analysis of 8085 Addressing Modes in an Industrial Temperature Control Unit

Sample temperatures (°C)

temperature = [28, 35, 42, 38, 45]

print("Industrial Temperature Control Unit")

```
print("-----------------------------------")

# Check each temperature
for temp in temperature:
 if temp > 40:
 print("Temperature:", temp, "°C -> FAN ON")
 else:
 print("Temperature:", temp, "°C -> FAN OFF")
```

Output:

```
Industrial Temperature Control Unit
-----------------------------------
Temperature: 28 °C -> FAN OFF
Temperature: 35 °C -> FAN OFF
Temperature: 42 °C -> FAN ON
Temperature: 38 °C -> FAN OFF
Temperature: 45 °C -> FAN ON
```

Using the 8085 Instruction SetEffectively

To ensure reliable operation, the following instructionsshould be used correctly: **Instruction Purpose**

LDA Load sensor data from memory **STA** Store processed temperature value

MOV Transfer data between registers

MVI Load fixed threshold values

CMP Compare temperature with reference value

JZ, JC, JNZ Decisionmaking based on comparison

IN Read data frominput ports(sensorinterface)

OUT Send controlsignalsto actuators

HLT Stop execution after completing tasks(ifrequired)

MeasurestoEnsure Accurate and ReliableOperation

- Use the correct addressing mode for each operation.
- Initialize register pairs(HL,BC, DE) before indirect addressing.
- Verifymemory addresses before accessing sensor data.
- Use comparison instructions(**CMP**)to validate readings.
- Store updated values correctly using **STA**.
- Use conditional branching (**JC, JZ, JNZ**) for proper control decisions.

Conclusion

Addressing modes in the **8085 microprocessor** determine how data is accessed from registers and memory. Incorrect use of these modes can cause the processor to read wrong temperature values, leading to improper control of industrial equipment. By selecting the appropriate addressing mode and using instructions such as **LDA, STA, MOV, MVI, CMP,IN, OUT, and conditional jump instructions**, the temperature control unit can process sensor data accurately and operate reliably.